

Universidad Nacional Autónoma de México

FACULTAD DE CIENCIAS

Ciencias de la Computación



Criptografía y Seguridad

Práctica 1

Cifrados Clásicos

Aguirre Cruz Cassandra -

González Téllez Lorena - 321288952

Vázquez Rincón Oscar - 321228954

4 de septiembre de 2026

1. Introducción

La criptografía es la ciencia encargada de diseñar funciones o dispositivos capaces de transformar mensajes legibles o en claro a mensajes cifrados de tal manera que dicha transformación (cifrar) y su transformación inversa (descifrar) sólo pueden ser factibles con el conocimiento de una o más llaves. En la criptografía existen dos tipos de cifrados: los simétricos, en donde la misma llave se usa para cifrar y descifrar, y los asimétricos, en donde se utilizan dos llaves distintas. Esta práctica se centra en los cifrados simétricos.

Los cifrados clásicos son aquellos que han tenido un uso histórico masivo, pero que hoy en día no se consideran seguros, pues han surgido técnicas modernas más sofisticadas a lo largo del tiempo.

Esta práctica tiene como objetivo implementar algunos de estos cifrados clásicos y aplicarlos para el cifrado y el descifrado de archivos. El desarrollo de la misma implica trabajar directamente a nivel de bytes y realizar corrimientos correspondientes según el cifrado empleado. El enfoque se centra en el cifrado César, el cifrado decimado, el cifrado afín y la codificación Base64. Para poder descifrar los archivos sin conocer la llave de antemano, se hace uso de las firmas de archivo o Magic Bytes. Se busca combinar este conocimiento con las propiedades matemáticas de los cifrados para deducir la llave correcta y recuperar los archivos originales.

La importancia de esta práctica radica en comprender la base y los cimientos de lo que es la criptografía moderna. Saber aplicar estos cifrados clásicos permite construir bases sólidas sobre el origen de la disciplina, aplicando congruencias y aritmética modular. Por otro lado, el enfrentarse a un escenario de descifrado, como el que plantea esta práctica, nos permite desarrollar conocimientos sobre el manejo de bytes y las firmas de archivo.

2. Desarrollo

El desarrollo de esta práctica consta de distintas etapas: los cifrados, las firmas de los archivos y el manejo de bytes.

2.1. Cifrados

Los cifrados implementados en esta práctica se encuentran en el archivo `cifrados_clasicos.py`. A continuación se describe el procedimiento de desarrollo de cada uno.

Cifrado César

El cifrado César es un cifrado de sustitución. De acuerdo a [1], si en un mensaje original se encuentra el símbolo de índice i de un alfabeto de n símbolos y al cifrarlo se le substituye por el símbolo de índice j del alfabeto desplazado de k lugares a la derecha, entonces podemos simbolizar el cifrado como:

$$j \equiv (i + k) \pmod{n}$$

Dado que en esta práctica haremos el manejo de archivos a nivel de bytes, el alfabeto que usaremos está compuesto por los valores de los bytes, es decir, $n = 256$ y los caracteres son los valores enteros en el rango $[0, 255]$. La llave k es un entero en el rango $[0, 255]$.

- **Cifrado:** La función `cesar_cifrar(data, k)` recibe la secuencia de bytes `data` y la llave entera k . Se itera sobre todos los bytes del mensaje (de acuerdo a la definición anterior) y se construye el resultado como un objeto `bytes`:

```
resultado = bytes((byte + k) % 256 for byte in data)
```

- **Descifrado:** La función `cesar_descifrar(data, k)` aplica lo inverso del cifrado y para ello se resta k a cada byte del mensaje (regresando el mismo número de lugares que se avanzó en el cifrado). De manera análoga la implementación es:

```
resultado = bytes((byte - k) % 256 for byte in data)
```

Inverso modular

Como parte de los cifrados Decimado y Afín es necesario calcular el inverso modular de la llave $a \in \mathbb{Z}_{256}$ respecto del módulo 256. El inverso modular x de a módulo m es el entero que satisface:

$$(a \cdot x) \equiv 1 \pmod{m}$$

La función `inverso_modular(a, m)` se calcula mediante el algoritmo de Euclides extendido:

```
m_original = m
x0 = 1
x1 = 0
while m != 0:
    cociente = a // m
    a, m = m, a % m
    x0, x1 = x1, x0 - cociente * x1
if a != 1:
    raise ValueError("Si no existe el inverso modular")
return x0 % m_original
```

A continuación veremos que su uso es imperativo en los otros cifrados.

Cifrado Decimado

Este esquema es similar al cifrado César, pero ahora el símbolo que reemplaza al i -ésimo es obtenido multiplicando i por algún número, es decir, el i -ésimo símbolo es reemplazado en el texto cifrado por el j -ésimo con $j = (ki) \pmod{n}$. Para que el cifrado sea válido en nuestros bytes y cumpla con ser una función biyectiva, es necesario elegir el factor de decimación de modo que sea primo relativo a $n = 256$.

- **Cifrado:** La función `decimado_cifrar(data, a)` multiplica cada byte del mensaje por la llave a y reduce módulo 256. La lógica es la siguiente:

1. Realizamos $a \pmod{256}$ para que la llave esté en el rango $[0, 255]$.
2. Verificamos que $\gcd(a, 256) = 1$ llamando a `inverso_modular(a, 256)`. Esto nos asegura de que la transformación sea biyectiva. De no cumplirse, no sería válido el cifrado porque no nos asegura que la función de cifrado sea biyectiva y por tanto el descifrado no sería posible.
3. Aplicamos la transformación de multiplicación byte a byte.

```
a = a % 256
inverso_modular(a, 256)
resultado = bytes((byte * a) % 256 for byte in data)
```

- **Descifrado:** La función `decimado_descifrar(data, a)` invierte el cifrado. Contrario al cifrado César, se multiplica por el inverso modular de a módulo 256 para recuperar el mensaje original. La lógica implementada es:

1. Aplicamos a (mód 256) para que esté en el rango $[0, 255]$.
2. Calculamos el inverso modular de a mediante `inverso_modular(a, 256)`.
3. Aplicamos la transformación de multiplicación inversa byte a byte.

```
a = a % 256
inverso = inverso_modular(a, 256)
resultado = bytes((byte * inverso) % 256 for byte in data)
```

Observación: Para que el cifrado sea reversible, a debe ser coprimo con 256, es decir, $\gcd(a, 256) = 1$. En el programa se define `LLAVES` como el conjunto de llaves válidas. En el intervalo $[1, 255]$ hay exactamente $\phi(256) = 128$ valores válidos.

Cifrado Afín

Este cifrado consiste de una combinación de los dos cifrados anteriores donde la regla de sustitución está dada por $j = (ri + k) \pmod n$, donde $i, r, k \in \{0, \dots, n-1\}$. Recordando que para nuestro caso $n = 256$: $i, r, k \in \{0, \dots, 255\}$. Igual que en el cifrado Decimado, también necesitamos que la transformación sea biyectiva, y por tanto, r debe ser coprimo con $n = 256$.

- **Cifrado:** La función `afin_cifrar(data, a, b)` aplica a cada byte la transformación que acabamos de mencionar. La lógica se sigue de:

1. Nos aseguramos que las llaves estén en $\{0, \dots, 255\}$: $a \pmod{256}$, $b \pmod{256}$.
2. Verificamos que $\gcd(a, 256) = 1$ para que la función de cifrado sea biyectiva.
3. Aplicamos la transformación definida byte a byte.

```
a = a % 256
b = b % 256
inverso_modular(a, 256)
resultado = bytes((a * byte + b) % 256 for byte in data)
```

- **Descifrado:** La función `afin_descifrar(data, a, b)` invierte la transformación afín. Considerando que en el cifrado $j = (a \cdot i + b) \pmod{256}$ se despeja i que es el dato original que queremos recuperar mediante la siguiente expresión:

$$i = (a^{-1} \cdot (j - b)) \text{ mód } 256$$

De modo que la lógica de descifrado está dada por:

1. Obtenemos los valores de las llaves a mód 256 y b mód 256.
2. Calculamos el inverso modular a^{-1} de a módulo 256.
3. Aplicamos la transformación inversa byte a byte.

```
a = a % 256
b = b % 256
inverso = inverso_modular(a, 256)
resultado = bytes((inverso * (byte - b)) % 256 for byte in data)
```

Base64

En esta codificación se convierte la secuencia de bytes en una secuencia de caracteres (en este caso ASCII), fue implementada en el archivo `base64.py`. La idea es que tres valores de byte de datos se codifican en cuatro valores Base64.

Primero es importante mencionar que el alfabeto de 64 caracteres se definió por el alfabeto:

```
CARACTERES = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"
```

Cada posición $i \in [0, 63]$ corresponde a un bloque de 6 bits con valor i .

- **Codificación:** La función `base64_encode` recibe una cadena binaria y aplica el siguiente procedimiento:

1. Determinar cuántos bytes completos de 3 caracteres forman la entrada. Determinando si la longitud es o no múltiplo de 3, se calcula cuántos caracteres de relleno son necesarios.

```
completo = len(data) % 3
p = 0 if completo == 0 else (3 - completo)
```

2. Se rellena hasta que la longitud sea múltiplo de 3, y luego cada byte se convierte a su representación binaria.

```
data_padded = data + b"\x00" * p
bits = "".join(format(b, "08b") for b in data_padded)
```

3. Recorremos la cadena binaria en bloques de 6 bits y se transforman a enteros en base 2 (como índices) para obtener el caracter correspondiente Base64 usando el alfabeto ya mencionado.

```
res = "".join(CARACTERES[int(data[i:i+6], 2)]
              for i in range(0, len(data), 6))
```

4. Finalmente, si $p > 0$, sustituimos los últimos p caracteres por '='.

```
if p > 0:
    res = res[:-p] + ("=" * p)
```

- **Decodificación:** Para poder decodificar el procedimiento anterior, se implementó la función `base64_decode` que recibe la cadena obtenida en Base64 y realiza el siguiente procedimiento para recuperar los bytes:

1. Mediante un diccionario se asocia cada caracter del alfabeto el índice numérico de 6 bits.

```
BUSQUEDA = {c: i for i, c in enumerate(CARACTERES)}
```

2. Se eliminan los caracteres de relleno '=' de la cadena.

```
padding = encoded.count("=")
word = encoded.replace("=", "")
```

3. Se transforman los caracteres a los índices correspondientes.

```
vals = [BUSQUEDA[c] for c in word if c in BUSQUEDA]
```

4. Se transforma a múltiplos de 4 rellenando con ceros, esto porque cada 4 bloques de 6 bits forman exactamente $4 \times 6 = 24$ bits (3 bytes).

```
while len(vals) % 4 != 0:
    vals.append(0)
```

5. Con desplazamientos bit a bit, por cada bloque de 4 valores se une un entero de 24 bits, lo que nos permite formar los 3 bytes que lo formaron desde un inicio:

```
result = bytearray()
for i in range(0, len(vals), 4):
    n = (vals[i] << 18) | (vals[i+1] << 12) | (vals[i+2] << 6) | vals[i+3]
    result.append((n >> 16) & 0xFF)
    result.append((n >> 8) & 0xFF)
    result.append(n & 0xFF)
```

6. Se eliminan los bytes de relleno que habíamos añadido en la codificación.

```
if padding:
    del result[-padding:]
```

2.2. Firmas de Archivos

Para esta etapa de la práctica se implementó el archivo `magic_bytes.py`, cuyo propósito es identificar el formato real de un archivo a partir de su contenido binario. Dado que las extensiones de los archivos pueden ser alteradas, los formatos binarios incluyen al inicio del archivo una secuencia fija de bytes que permite identificar el formato.

Como parte de la implementación se definió el diccionario `MAGIC_BYTES_HEX` que mapea la representación hexadecimal de cada firma al nombre de su formato:

```
MAGIC_BYTES_HEX = {
    "89504e470d0a1a0a": "png",
    "ffd8ff":           "jpg",
    "424d":             "bmp",
    "494433":           "mp3",
    "fffb":             "mp3",
    "4f676753":         "ogg",
    "0000001866747970": "mp4",
    "1a45dfa3":         "mkv",
    "504b0304":         "docx",
    "25504446":         "pdf",
    "65707562":         "epub",
}
RIFF = "52494646"
```

Además, se definió la constante `RIFF = "52494646"` para manejar de forma especial los formatos WAV y AVI, que comparten los mismos 4 bytes iniciales 52 49 46 46.

La función `detectar_formato(data)` recibe el contenido binario de un archivo y determina su formato. El procedimiento es el siguiente:

1. Se extraen los primeros 16 bytes del archivo, tanto para flujos de datos (objetos con método `read`) como para secuencias de bytes:

```
if hasattr(data, 'read'):
    start_pos = data.tell() if hasattr(data, 'tell') else 0
    bytes_data = data.read(16)
```



```
        if hasattr(data, 'seek'):
            data.seek(start_pos)
    else:
        bytes_data = data[:16]
```

2. Los bytes extraídos se convierten a su representación hexadecimal en minúsculas:

```
hexbytes = bytes_data.hex()
```

3. Si los primeros 4 bytes corresponden a RIFF (52494646), no es posible distinguir WAV de AVI y se inspeccionan los bytes 8 a 11 del archivo. Si no coinciden con alguno, devuelve None:

```
if hexbytes.startswith(RIFF) and len(bytes_data) >= 12:
    subtype = bytes_data[8:12]
    if subtype == b"WAVE": return "wav"
    if subtype == b"AVI ": return "avi"
    return None
```

4. Se itera sobre el diccionario MAGIC_BYTES_HEX. Al encontrar una coincidencia devuelve el formato correspondiente:

```
for magic_hex, formato in MAGIC_BYTES_HEX.items():
    if hexbytes.startswith(magic_hex):
        return formato
return None
```

5. Si ninguna firma coincide, la función retorna None.

2.3. Manipulación de bytes

El archivo `archivos_bytes.py` fue implementado para manejar las operaciones de entrada a nivel de bytes.

La función `leer_archivo(ruta)` recibe la ruta de un archivo y devuelve su contenido completo como un objeto `bytes`. Para ello comprueba que el archivo existe en la ruta recibida y luego se abre el archivo en modo binario.

El uso del modo binario es necesario ya que los cifrados operan directamente sobre los valores numéricos de cada byte (0–255).

2.4. Descifrador de Archivos

Después de todas las implementaciones anteriores se implementó el programa `main.py` que integra el proceso de descifrado de un archivo del cual no conocemos su cifrado ni sus llaves. Por lo que para poder descifrarlo se aplicó fuerza bruta probando algoritmo por algoritmo con todas las posibles llaves, esto hasta encontrar el correcto si es que fue cifrado con alguno de los que implementamos.

Para mantener el código organizado se definieron dos funciones auxiliares:

- `guardar(datos, nombre_base, formato)`: Para guardar los archivos que se descifren correctamente.
- `resultado(cifrado, llave, formato, ruta)`: Para imprimir un resumen del descifrado.

Como mencionamos antes, aplicamos fuerza bruta para cada cifrado. Para hacerlo más efectivo, en cada intento se opera sobre los primeros 16 bytes para verificar las firmas antes de descifrar el archivo completo. De esta manera, tenemos una función de fuerza bruta por cada cifrado, en todas estas funciones se intenta descifrar los primeros 16 bytes y si coincide con alguna firma, se descifra el archivo completo (se guarda). Lo que cambia entre ellas es el espacio de llaves. Por lo que estas funciones son:

- **Base64** `base64_fuerza_bruta`: Interpreta los bytes como texto ASCII y los decodifica, no hay llaves para iterar.
- **César** `cesar_fuerza_bruta`: Itera sobre todas las posibles llaves $k \in \{0, 1, \dots, 255\}$.
- **Decimado** `decimado_fuerza_bruta`: Itera sobre todas las llaves coprimas con 256.
- **Afin** `afin_fuerza_bruta`: Itera sobre todas las posibles llaves (a, c) con a coprimo con 256.

Como resultado final, la función `descifrar_archivo(ruta_cifrado)` une toda la lógica de descifrado que hemos ido desarrollando:

1. Verificar que la ruta recibida corresponde a un archivo existente. Si no existe, terminar con error.
2. Leer el contenido binario completo con `leer_archivo`.
3. Iterar sobre la lista de métodos en orden:

```

METODOS_CIFRADO = [
    base64_fuerza_bruta,
    cesar_fuerza_bruta,
    decimado_fuerza_bruta,
    afin_fuerza_bruta,
]
for metodo in METODOS_CIFRADO:
    if metodo(datos, nombre_base):
        break

```

4. El primer método que retorne `True` detiene la búsqueda. De otro modo, se concluye que el archivo no fue cifrado con ninguno de los métodos implementados.

El programa se ejecuta desde la terminal de la siguiente forma:

```
python src/main.py <ruta_del_archivo_cifrado>
```

2.5. Preguntas

1. ¿Cuántos primos relativos hay en $\mathbb{Z}_{256}$?

Primero conviene recordar que dos enteros $a, b \in \mathbb{Z}$ son primos relativos si su máximo común divisor es 1, es decir, si $\gcd(a, b) = 1$.

Por otro lado, $\mathbb{Z}_m$ denota al conjunto de todas las clases de equivalencia diferentes módulo m . Donde cada clase de equivalencia se define como $[z]_m = \{x \in \mathbb{Z} \mid x \equiv z \pmod{m}\}$. De este modo, cuando hablamos de $\mathbb{Z}_{256}$ nos referimos a un conjunto con 256 elementos, es decir:

$$\mathbb{Z}_{256} = \{[0]_{256}, [1]_{256}, [2]_{256}, \dots, [255]_{256}\}$$

Después de estos recordatorios, para poder contar los primos relativos en $\mathbb{Z}_{256}$ podemos usar la función indicatriz de Euler. Si $n \in \mathbb{N}$, entonces $\phi(n)$ se define como el número de enteros positivos menores o iguales a n y coprimos con n , es decir, formalmente se puede definir como:

$$\phi(n) = |\{m \in \mathbb{N} \mid 1 \leq m \leq n \wedge \gcd(m, n) = 1\}|$$

Dentro de las propiedades que nos ofrece la función indicatriz de Euler encontramos que:

$$\phi(p^k) = (p - 1)p^{k-1} \text{ si } p \text{ es un número primo y } k \in \mathbb{N}$$

Dado que queremos contar los primos relativos en $\mathbb{Z}_{256}$, conviene ver que $256 = 2^8$. Por lo que $p = 2$ (pues 2 es primo) y $k = 8 \in \mathbb{N}$ en la propiedad que acabamos de enunciar. Aplicando la fórmula:

$$\phi(2^8) = (2 - 1)2^{8-1} = 1 \cdot 2^7 = 128$$

Esto nos muestra que existen 128 primos relativos en $\mathbb{Z}_{256}$.

2. Sea $f : \mathbb{Z}_n \rightarrow \mathbb{Z}_n$ tal que $f(i) = k \cdot i$ para $i \in A$ con A un alfabeto cualquiera. ¿Qué se debe cumplir para que f sea una función biyectiva?

Por definición, para que f sea biyectiva debe cumplirse que sea inyectiva y sobreyectiva al mismo tiempo.

Inyectividad: Para que f sea inyectiva debe cumplirse que no hay dos elementos del dominio que tengan la misma imagen, es decir, para $a, b \in \mathbb{Z}_n$, si $f(a) = f(b)$, entonces $a = b$.

Supongamos primero que para $a, b \in \mathbb{Z}_n$, $f(a) = f(b)$, es decir, $[k \cdot a]_n = [k \cdot b]_n$. Recordando que dos clases de equivalencia son iguales si y sólo si sus representantes son congruentes módulo n ($[x]_n = [y]_n \iff x \equiv y \pmod{n}$) tenemos que $ka \equiv kb \pmod{n}$, lo cual es equivalente a:

$$k(a - b) \equiv 0 \pmod{n}$$

es decir, $n \mid k(a - b)$ (por definición de congruencia). Aquí es importante mencionar que buscamos cancelar el factor k para que se cumpla que $n \mid (a - b)$, que por definición de congruencia implicaría $a \equiv b \pmod{n}$.

Para este propósito, recurrimos al Lema de Euclides donde si $n \mid k \cdot m$ y $\gcd(k, n) = 1$, entonces $n \mid m$.

De esta manera, si se cumple que $\gcd(k, n) = 1$, podemos concluir que $n \mid (a - b)$, es decir, $a \equiv b \pmod{n}$, lo que en $\mathbb{Z}_n$ significa que $[a]_n = [b]_n$, por lo tanto $a = b$ y f es inyectiva.

Observación: La condición $\gcd(k, n) = 1$ es la hipótesis que necesitamos para que el argumento sea válido. Si en cambio $\gcd(k, n) = d > 1$, entonces existe $m = n/d$ con $0 < m < n$ tal que $k \cdot m \equiv 0 \pmod{n}$. Esto se debe a que $d \mid k$ (pues $d = \gcd(k, n)$), por lo que $k = d \cdot q$ para algún $q \in \mathbb{Z}$, y entonces $k \cdot m = d \cdot q \cdot (n/d) = q \cdot n$, que es claramente múltiplo de n . Esto implica $f(m) = [k \cdot m]_n = [0]_n = f(0)$ con $m \neq 0$, por lo que f no sería inyectiva y, por tanto, no sería biyectiva.

Sobreyectividad: Para que f sea sobreyectiva debe cumplirse que todo elemento del codominio $\mathbb{Z}_n$ es imagen de algún elemento del dominio, es decir, para todo $j \in \mathbb{Z}_n$ existe $i \in \mathbb{Z}_n$ tal que $f(i) = j$.

Ahora bien, si $\gcd(k, n) = 1$, entonces k es invertible en $\mathbb{Z}_n$, es decir, existe $k^{-1} \in \mathbb{Z}_n$ tal que $k \cdot k^{-1} \equiv 1 \pmod{n}$. Dado un elemento cualquiera $j \in \mathbb{Z}_n$, basta tomar $i = k^{-1} \cdot j \in \mathbb{Z}_n$, pues:

$$f(i) = k \cdot (k^{-1} \cdot j) = (k \cdot k^{-1}) \cdot j \equiv 1 \cdot j \equiv j \pmod{n}$$

Así, todo elemento de $\mathbb{Z}_n$ tiene preimagen y f es sobreyectiva.

$\therefore$ La condición necesaria y suficiente para que $f(i) = k \cdot i$ sea una función biyectiva en $\mathbb{Z}_n$ es que $\gcd(k, n) = 1$, es decir, que k y n sean primos relativos.

3. ¿Cuántas posibles combinaciones no triviales existen para cifrar bytes con César, Decimado y Afín?
 - **César:** 255 combinaciones no triviales (256 en total, menos la clave trivial $k = 0$).
 - **Decimado:** 127 combinaciones no triviales (128 números impares coprimos con 256, menos la clave trivial $a = 1$).
 - **Afín:** 32,767 combinaciones no triviales ($128 \times 256 = 32,768$ en total, menos la clave trivial $a = 1, b = 0$).
4. ¿Por qué el sistema de archivos de UNIX, aunque un archivo tenga una extensión diferente (o incluso no tenga), sigue reconociendo al archivo original?
 - UNIX inspecciona el interior del archivo usando los primeros bytes para leer su firma digital, la cual indica el tipo de archivo real. También los metadatos y datos físicos residen todavía. El nombre del archivo y su extensión son solo una etiqueta visual para el usuario que apunta a este.
5. ¿Por qué los archivos descifrados tienen exactamente el mismo tamaño que antes de cifrar, pero no pudimos leerlos? ¿Por qué no tuvimos que agregar/quitar nada?
6. Ya que Base64 no es un cifrado, sino codificación, ¿en qué casos podemos usarlo?

3. Conclusiones

Como resultado de esta práctica consideramos importante lo que aprendimos sobre estos cifrados porque forman parte de la historia en la Criptografía y son la base del desarrollo de cifrados más complejos que se usan hoy en día. Por otra parte, consideramos que el uso de estos cifrados ya no es tan recomendado debido a la simplicidad de los mismos, pues son únicamente algoritmos de sustitución y como hemos analizado, conociendo el espacio de las llaves puede ser descifrado. Así también, fue enriquecedor el análisis del álgebra que sustenta los cifrados, pues nos ayuda a entender mejor como funcionan.

Un reto dentro del desarrollo fue acotar el intento de descifrado por fuerza bruta, pues al inicio tomaba demasiado tiempo y no concluían algunos archivos. Por ello se decidió iniciar con las firmas de los archivos (los primeros 16 bytes) para poder ir avanzando un poco más rápido.

Así también, considerar el caso de los formatos WAV y AVI por separado fue algo que al inicio contemplamos integrarlos en un mismo diccionario, pero debido a que se terminarían sobreescribiendo sólo se identificaría uno de los dos, por ello fue necesario separarlos del diccionario y aplicar una lógica distinta por aparte.

Por último, otro reto enfrentado fue al momento de implementar la codificación Base64, nuestra versión inicial del programa tardaba mucho tiempo y se fue modificando poco a poco para que pudiera terminar el programa. Antes se habían implementado muchos ciclos anidados que retardaban mucho el proceso (pues funcionaba con entradas pequeñas pero conforme aumentaba, tardaba mucho más), por lo que nos atoramos en esta parte para que se pudiera decodificar exitosamente.

Como resultado pudimos implementar el programa antes desarrollado y aprender sobre estos cifrados clásicos y la codificación Base64.

Referencias

- [1] Galaviz Casas, J. *Introducción a la Criptología*. 2010. Disponible en: https://drive.google.com/file/d/1FxSX0-ymteWoxSPeR_3w7Mxd2H11bIRe/view
- [2] Jens@Fivesec. *Revisiting the Basics: Implementing Base64 from Scratch in pure C*. Medium, 2024. Disponible en: https://medium.com/@js_9757/revisiting-the-basics-implementing-base64-from-scratch-in-pure-c-c6985fb08420
- [3] Base64 Algorithm Description. Disponible en: https://www.sunshine2k.de/articles/coding/base64/understanding_base64.html
- [4] Magic Bytes. Disponible en: https://en.wikipedia.org/wiki/List_of_file_signatures
- [5] Identificador de Archivos (Magic Bytes). Disponible en: https://tool.lu/es_ES/magicbytes/